

MediClaim Orchestrator

Every Feature: What It Does and How

A simple speaking script for each part of the system

Multi-Agent Cashless and Reimbursement Claim Processing

Read the 'Say this' boxes out loud

October 2026

Contents

Start here: the one-line answer.....	3
Part 1: The hospital side.....	4
1. Uploading documents.....	4
2. Virus scan and quality check.....	4
3. Reading the documents (OCR and fields).....	4
4. Hiding personal data (privacy).....	5
5. The document checklist.....	5
6. Building the claim (CrewAI ClaimFlow).....	6
7. Estimating the payout before sending.....	6
8. The 12 checks (V01 to V12).....	7
9. Review, sign-off and submission.....	7
Part 2: The insurer side.....	9
10. Verifying the claim (CrewAI VerificationFlow).....	9
11. Calculating the amount (calc engine).....	9
12. The decision gate.....	10
13. Queries: asking and answering.....	11
14. Decision and payment.....	12
15. The audit trail.....	12
Part 3: What works in the background.....	13
16. Policy search (RAG).....	13
17. Keeping the agents honest (guardrails and the bridge).....	13
18. Workflows and automation (n8n).....	14
19. Login and roles.....	14
20. Live updates on screen.....	14
21. Offline mode.....	15
22. One-command build and run.....	15
23. Test data and evaluation.....	16
Quick summary table.....	17

Start here: the one-line answer

Say this first

"Our system takes a patient's hospital documents and turns them into a correct insurance claim, sends it to the insurer, checks it, decides on it and pays it. AI agents do the reading and writing, normal code does the rules and the money, and people approve the important decisions."

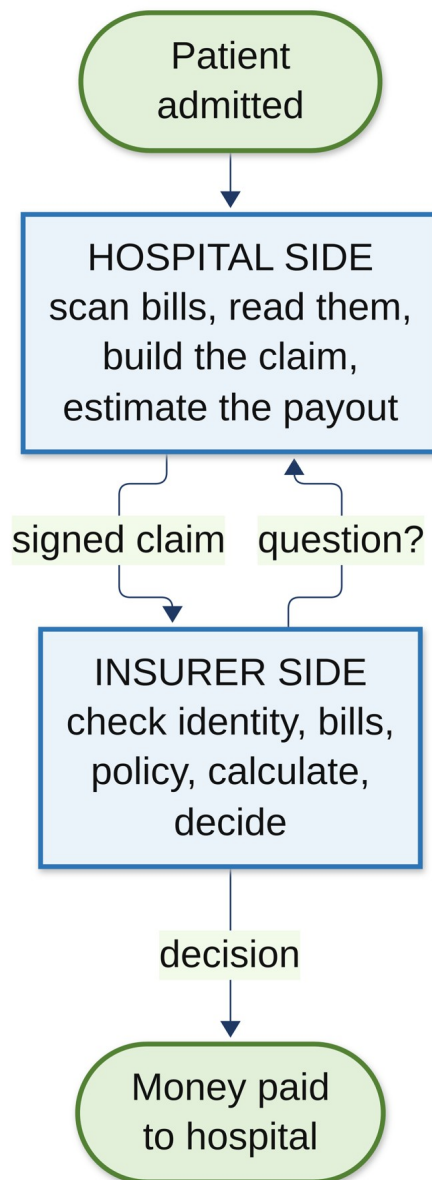


Figure 1: The big picture

The features below follow the life of one claim, from upload to payment. After that come the features that work in the background: search, safety, workflows, the demo and testing.

Part 1: The hospital side

1. Uploading documents

What it does: Lets the hospital desk create a case for a patient and upload the bills, discharge summary, lab reports, ID card and policy card.

How it does it:

1. The desk opens the hospital web page (Next.js) and creates a case.
2. They drag in the PDFs or photos.
3. The hospital API saves each file in MinIO (our file store), encrypted.
4. The API then tells n8n 'a document was uploaded' through a webhook, and n8n takes it from there.

Say this

"The desk staff create a case and upload the documents on a web page. Each file is saved safely in our file store, and the system automatically starts processing it."

2. Virus scan and quality check

What it does: Makes sure every file is safe and readable before anyone uses it.

How it does it:

1. ClamAV, an antivirus, scans the file. If it's not clean, the process stops right there.
2. The vision service (OpenCV) checks the image: is it blurry, too dark, cut off?
3. If the quality is too poor, the document is marked 'needs review' so the desk can upload a better copy.

Say this

"Before anything reads a file, it's virus-scanned. Then we check the photo quality. A blurry bill causes queries later, so we catch it right at the start."

3. Reading the documents (OCR and fields)

What it does: Turns a scanned page into useful data: what kind of document it is, and the important fields like name, dates and amounts.

How it does it:

1. Poppler turns each PDF page into an image.
2. Tesseract (OCR) reads the text from the image.
3. The document pipeline classifies the document: bill, discharge summary, ID card, policy card and so on.

4. It picks out the fields. Simple rules go first (for example a policy number pattern), and the AI model only helps when the rules can't find a field.
5. Every field gets a confidence score. The reading is done in two passes so they can be compared.

Example: From a policy card it pulls out the policy number, member ID, valid from and to dates, product name and sum insured.

Say this

"We read each document with OCR, work out what type it is, and pull out the fields we need. Rules go first and AI only fills the gaps, and every field has a confidence score so we know how sure we are."

4. Hiding personal data (privacy)

What it does: Makes sure the AI never sees more personal information than it needs.

How it does it:

1. Presidio, plus our own PII guard, finds names, phone numbers, Aadhaar-like numbers and similar details.
2. These are masked (hidden) before the text goes to any AI model.
3. The guard sits in front of every single AI call, so it can't be skipped.

Say this

"Health data is private. So before any text reaches the AI, personal details like names and phone numbers are hidden. And the AI runs on our own laptop anyway, so nothing leaves the machine."

5. The document checklist

What it does: Shows the desk which documents are present and which are still missing for this kind of claim.

How it does it:

1. The hospital API knows which documents each type of claim needs.
2. Every time a document is classified, the API updates the checklist.
3. n8n tells the screen to refresh, so the desk sees it straight away.
4. When everything is there, the case moves to 'docs complete' and the claim build starts.

Say this

"There's a live checklist on screen. The moment all the required documents are in, the system moves on and starts building the claim by itself."

6. Building the claim (CrewAI ClaimFlow)

What it does: Turns all the bills into one clean claim: every line in the right category, with correct totals.

How it does it:

1. The hospital API starts the CrewAI Flow called ClaimFlow.
2. The flow first checks: is this a new claim, or a repair of an existing draft? It takes the right path.
3. Code collects all the bill lines and adds up the totals. AI is not used for adding.
4. Lines that code can't place are sent to the **category mapper agent**, which puts each one in a group like room, medicine, lab test or surgery.
5. A guardrail checks the agent's answer: only allowed categories are accepted. A wrong answer goes back to the agent once with feedback.
6. If the build gets stuck, an n8n workflow restarts it every 5 minutes.

Say this

"The claim is built by a CrewAI Flow. Code collects the lines and does the totals, and an agent only steps in to sort lines that the rules can't place. A guardrail checks the agent only uses allowed categories."

7. Estimating the payout before sending

What it does: Tells the hospital roughly how much the insurer will pay, and how much the patient will pay, before the claim is even sent.

How it does it:

1. The flow finds the patient's policy from the policy card.
2. The RAG service searches the policy wording for the right paragraphs.
3. The **policy estimator agent** reads them and finds the rules: room rent limit, ICU limit, co-pay, sub-limits. It must quote the exact sentence.
4. A guardrail checks that the quote really is in the wording and contains the number. A code parser double-checks and wins if they disagree.
5. The calculation engine does the maths.
6. The estimate is shown on an 'estimate card' on screen. It's only advice and never stops the claim.

Example: Bill ₹55,800, minus ₹800 non-medical charge, minus 10% co-pay → estimated payout ₹49,500, patient pays ₹6,300.

Say this

"Before sending, we estimate the payout. An agent reads the patient's own policy and finds the co-pay and room rent rules, with the exact sentence as proof, and the calculator does the maths. The hospital knows the patient's share in advance, so there are no surprises at discharge."

8. The 12 checks (V01 to V12)

What it does: Catches mistakes before the insurer sees them, so there are fewer queries.

How it does it:

1. The hospital API runs 12 fixed checks on the draft claim.
2. They check things like: do the lines add up to the total (V01, V02), quantity × price (V03), does each line point to a real document page (V04), do the dates match the stay (V05), do the name and date of birth match (V06), is there a diagnosis code (V07), do the bill totals match (V08), duplicate lines (V09), more than pre-approved (V10), room rent over the limit (V11), and are all documents from this case and virus-free (V12).
3. Problems are shown as blockers or warnings.

Say this

"We run 12 checks, like do the totals add up, do the dates match, is the name right. Most queries happen because of small mistakes, so we catch them before the claim leaves the hospital."

9. Review, sign-off and submission

What it does: A person checks the claim, and then it's sent to the insurer safely.

How it does it:

1. The billing officer sees the claim, the check results and the estimate, and can edit anything.
2. They sign off and press submit. The AI never submits by itself.
3. The claim goes into an outbox, so even if the network fails it's retried until it gets through.
4. It's signed with HMAC so nobody can change it on the way, and it carries an idempotency key so it's processed only once.
5. n8n watches: if the insurer doesn't acknowledge in 15 minutes a banner appears, and after 8 hours an alert is raised.

Say this

"A real person always signs off. Then the claim is sent signed, so it can't be tampered with, and through an outbox, so it's never lost. If the insurer doesn't respond, the system raises an alert."

Part 2: The insurer side

10. Verifying the claim (CrewAI VerificationFlow)

What it does: Checks the claim on the insurer's side: right patient, genuine bills, covered by the policy, correct mapping for the calculator.

How it does it:

1. When the claim arrives, verification starts automatically (n8n workflow 10).
2. First the documents are fetched and completeness is checked. This is code only.
3. Code works out the hard facts: name match score, does the date of birth match, does the policy match, do the bill totals add up, are there duplicates, is there a stamp.
4. Then the agents run, in order: **identity** (is this the right patient?), **authenticity** (do the bills look genuine?), **coverage** (what does the policy say? It quotes the clauses using RAG), and **calculation mapper** (which policy group does each bill line belong to?).
5. A **supervisor** agent reads all of it and writes a short summary for the reviewer.
6. The agents can explain, but they can't overrule the facts. If code says the name doesn't match, it doesn't match.

Say this

"On the insurer side, code first works out the hard facts, like does the name match. Then CrewAI agents check identity, authenticity and coverage, and a supervisor agent writes a summary for the reviewer. The agents explain, but they can never overrule the facts."

11. Calculating the amount (calc engine)

What it does: Works out the exact amount the insurer should pay.

How it does it:

1. It's plain Python using exact decimal numbers, not floating point and not AI.
2. It removes items that aren't covered, like non-medical charges.
3. It applies the room rent limit. If the patient took a costlier room, other room-linked charges are reduced in the same ratio (proportionate deduction).
4. It applies sub-limits (for example cataract ₹40,000), deductibles and co-pay, and checks the sum insured left.
5. Every rupee removed comes with a reason, so the decision can be explained.

Say this

"The amount is calculated by plain code with exact decimals, never by AI. It applies room rent limits, co-pay and sub-limits, and every deduction has a reason. In 300 test claims it matched an independent calculator 100 percent."

12. The decision gate

What it does: Decides who approves the claim: the system, one person, or two people.

How it does it:

1. If any check raised a blocker or warning, or any step didn't work properly, it always goes to a person.
2. If the claim is clean and up to ₹50,000, the system can approve it automatically, and records why.
3. From ₹50,000 to ₹5,00,000, one approver decides.
4. Above ₹5,00,000, two approvers must agree.
5. n8n pops up a notification for the approvers.

Say this

"The decision gate decides who approves. Small, clean claims up to fifty thousand can be auto-approved, bigger ones need one approver, and above five lakh, two. Anything doubtful always goes to a person. In our tests there were zero wrong auto-approvals."

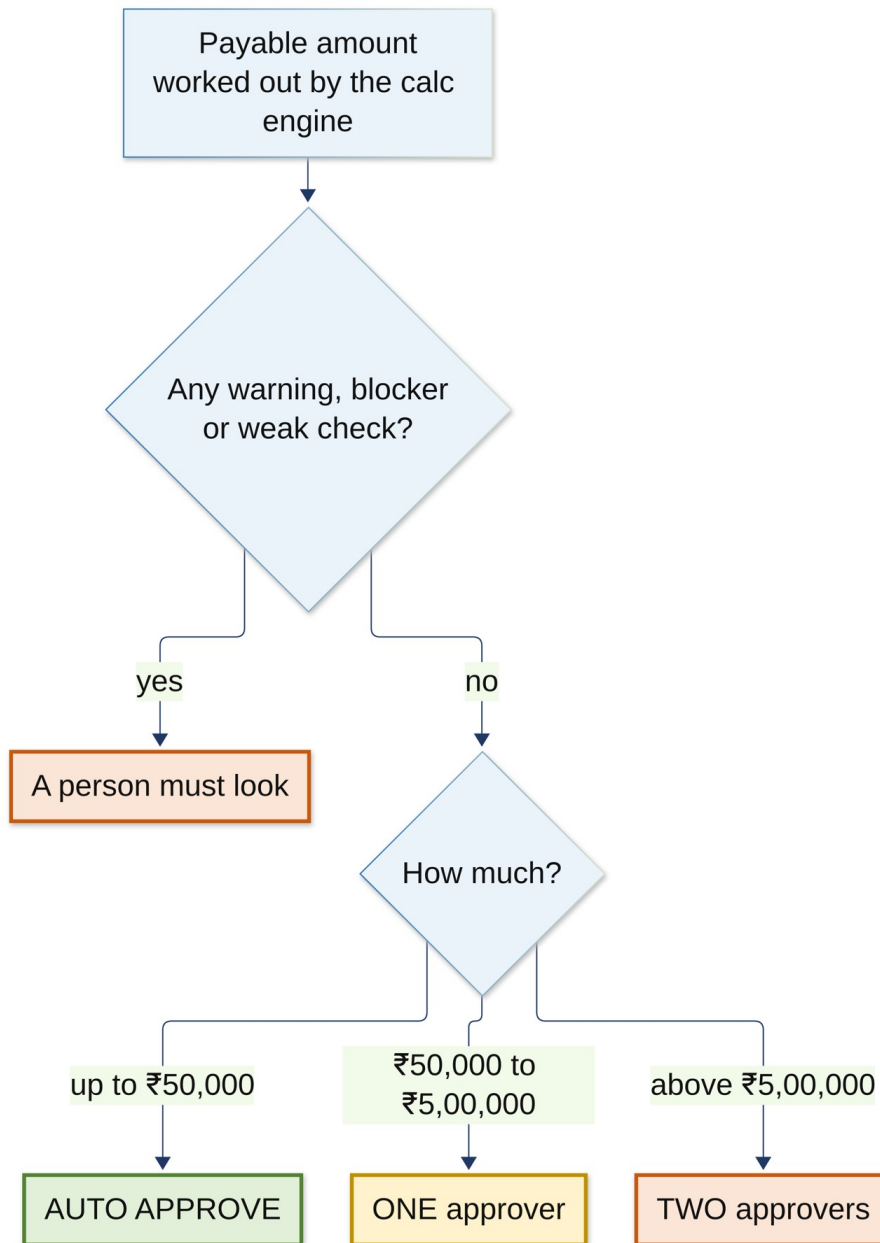


Figure 2: The decision gate

13. Queries: asking and answering

What it does: Handles the back-and-forth when something is missing or unclear.

How it does it:

1. If verification finds something missing, the insurer's **query drafter agent** writes a clear query to the hospital.
2. On the hospital side, n8n starts the **query triage agent**, which works out what kind of query it is: send a document, or explain something?
3. Then the **reply writer agent** drafts an answer using only facts from the case, with citations. Grounding rules (G01 to G08) check it doesn't invent anything.
4. The billing officer approves the reply before it goes.

5. On the insurer side, the **triage agent** checks the reply actually answers the query, and verification runs again.
6. After 3 rounds, the claim goes to a senior reviewer (escalation).

Say this

"If something's missing, the insurer's agent writes a query. On the hospital side one agent sorts it and another drafts a reply using only real facts, and a person approves it. After three rounds it goes to a senior reviewer, so it never loops forever."

14. Decision and payment

What it does: Tells the hospital the result and pays the money.

How it does it:

1. The decision (approved, partly approved or rejected) is sent back to the hospital as a signed callback.
2. If it's approved, n8n starts the payment through the bank simulator.
3. n8n checks every 30 seconds (up to 12 times) whether the payment went through, in case the bank's message gets lost.
4. The hospital gets a 'settled' message and the case is closed.

Say this

"Once decided, the result goes straight back to the hospital, the payment is sent, and the system keeps checking until it's confirmed. Then the case is closed."

15. The audit trail

What it does: Keeps a record of every step on both sides that nobody can secretly change.

How it does it:

1. Every important action is written to an audit table.
2. Each record contains a hash of the record before it, like links in a chain.
3. If someone edits an old record, the chain no longer matches.
4. Every night at 2 am an n8n workflow checks the whole chain.

Say this

"Every step is recorded, and each record is chained to the one before it with a hash, a bit like a blockchain. If anyone changes an old record, we can tell, and the system checks the whole chain every night."

Part 3: What works in the background

16. Policy search (RAG)

What it does: Finds the right part of a policy document so the agents can quote it.

How it does it:

1. Policy wordings are split into small pieces and stored in Qdrant.
2. Each piece is turned into numbers with nomic-embed-text, so we can search by meaning.
3. A search combines meaning-based search with keyword search, and a reranker puts the best matches on top.
4. Each side has its own collections and its own access token, so the hospital can't read the insurer's private rules.
5. If nothing good is found, the agent must say 'insufficient evidence' instead of guessing.

Say this

"When an agent needs a policy rule, it first searches the actual policy wording and quotes it. That's called RAG. It stops the AI from making things up, and if it can't find the answer, it has to say so."

17. Keeping the agents honest (guardrails and the bridge)

What it does: Makes sure the AI's answers are safe, correctly shaped and true to the facts.

How it does it:

1. Every agent talks to the model through our own bridge. The bridge hides personal data, repairs broken JSON, counts tokens, and stops calling the model if it keeps failing (circuit breaker).
2. Each answer is checked by Pydantic: is it the right shape?
3. Then a guardrail checks the content: are the categories allowed, is the quote real, are the facts grounded?
4. A bad answer goes back to the agent with feedback, once. If it fails again, the code's answer is used, or the step is marked 'degraded' and a person takes over.
5. CrewAI's memory is switched off, so no information leaks from one claim to another.

Say this

"Every agent answer is checked twice: once for shape and once for content. If it's wrong, the agent gets one more try with feedback. If it's still wrong, code takes over or a person decides. The AI never gets the final word."

18. Workflows and automation (n8n)

What it does: Runs the business process automatically: starting jobs, waiting, retrying, reminding and alerting.

How it does it:

1. There are 13 hospital workflows and 19 insurer workflows.
2. They are started by webhooks (something happened), clocks (every 2 or 5 minutes, every hour, 2 am nightly), crashes (error workflow) or other workflows (helpers).
3. Every webhook checks a secret or signature, replies straight away, and drops duplicates.
4. Failed API calls are retried after 2, 8 and 30 seconds.
5. The workflows are written in Python code and loaded automatically when n8n starts.

Say this

"n8n is our manager. It reacts to events, runs timers, waits, retries and sends reminders. But it never makes a decision. It only says when to do something, and the API decides what happens."

19. Login and roles

What it does: Makes sure only the right people and services can do each thing.

How it does it:

1. Keycloak handles login for desk staff, approvers and admins.
2. Each role can only do its own job. For example, only an approver can approve.
3. Services get their own tokens to call each other, and each one has its own database and file-store password with only the access it needs.

Say this

"Login and roles are handled by Keycloak, the industry standard. Desk staff, approvers and admins can each only do their own job, and even our services only get the access they need."

20. Live updates on screen

What it does: Shows changes on the screen straight away without refreshing.

How it does it:

1. When something happens (a document is classified, an approval is needed), the API publishes an event.
2. Redis carries the event, and the screen receives it instantly (server-sent events).
3. That's how the checklist, approval pop-ups and banners appear by themselves.

Say this

"The screens update live. When the checklist changes or an approval is needed, it just appears, because events are pushed through Redis to the browser."

21. Offline mode

What it does: Lets the whole system run even without the AI model.

How it does it:

1. Setting `CREW_LLM=rules` (hospital) or `INS_CREW_LLM=offline` (insurer) swaps the model for a rule-based stand-in.
2. The same agents, flows and guardrails still run, only the 'brain' is replaced.
3. `make demo-offline` runs the full claim in about 35 seconds.

Say this

"We also have an offline mode with no AI model at all. The same flows and checks run with rule-based stand-ins. It's a safe backup for the demo, and it proves the system doesn't fall apart if the model is down."

22. One-command build and run

What it does: Builds and starts the whole project with one command.

How it does it:

1. `make demo` checks your computer first: Docker, models, memory, free ports, PDF tools.
2. It creates all secrets, installs packages, starts the containers, sets up both databases, loads the policy documents into search, starts all services and both agent teams, and imports the workflows.
3. Then it runs real claims from upload to payment and prints a score card.
4. If a step fails, it says which one and where the log is. Running it again is safe.
5. `make demo-down` stops everything.

Say this

"The whole project starts with one command, make demo. It checks the machine, starts everything, runs real claims end to end, and prints a PASS or FAIL for every step."

23. Test data and evaluation

What it does: Proves the system works, with numbers.

How it does it:

1. A generator creates made-up but realistic patients, policies, bills and scanned documents. No real patient data is used.
2. We run 300 synthetic claims and measure each part against the right answer.
3. Hundreds of automatic tests also check every service, and browser tests click through the screens.

Say this

"We test on 300 made-up but realistic claims. The calculation matched an independent calculator 100 percent, bill mapping was 100 percent, there were zero wrong auto-approvals, and policy search found the right text 93 percent of the time. The one thing still to improve is exact quoting, at 61 percent against a target of 90."

Quick summary table

Feature	What it does	How (main tech)
Upload	Desk uploads documents	Next.js screen, hospital API, MinIO
Safety check	Virus and quality check	ClamAV, OpenCV vision service
Reading	OCR, classify, extract fields	Poppler, Tesseract, doc-pipeline (rules first, AI second)
Privacy	Hide personal data	Presidio, PII guard
Checklist	Show missing documents	Hospital API, n8n, live events
Claim build	Bill lines and totals	CrewAI ClaimFlow, category mapper agent
Estimate	Payout before sending	Policy estimator agent, RAG, calc engine
12 checks	Catch mistakes early	Hospital API rules V01-V12
Submit	Send safely	Outbox, HMAC, idempotency key
Verify	Identity, authenticity, coverage	CrewAI VerificationFlow, 5 agents
Calculate	Exact payable amount	Calc engine, exact decimals
Decide	Who approves	Decision gate: ₹50,000 / ₹5,00,000
Queries	Ask and answer	Query drafter, triage and reply writer agents
Pay	Send the money	n8n settlement workflow, bank simulator
Audit	Tamper-proof record	Hash chain, nightly check
Search	Find policy text	Qdrant, nomic-embed-text, reranker
Guardrails	Keep AI honest	Pydantic, guardrails, bridge
Automation	Run the process	n8n, 32 workflows
Login	Right access for each person	Keycloak
Run	One command	make demo, Docker Compose, uv

Closing line

"So in short: AI reads and writes, code does the rules and the money, people approve the important decisions, and the whole thing runs on one laptop with one command."